



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE CIENCIAS

**Reporte proyecto 1 (Implementación de protocolo
de chat utilizando sockets TCP).**

PRESENTA

Bruno Bernardo Soto Lugo
426008646

PROFESOR

Canek Peláez Valdés

AYUDANTES

Miriam Torres Bucio
Cynthia Lizbeth Sánchez Urbano

1. Introducción

El presente documento detalla el proceso de implementación de un protocolo que especifica una forma de comunicación chat. El sistema está compuesto por dos componentes principales: un cliente con interfaz gráfica desarrollado en **Kotlin** con el framework **Compose Multiplatform** y un servidor desarrollado en lenguaje **C** con conexión red usando sockets TCP/IP.

El objetivo principal del proyecto es implementar un protocolo de comunicación basado en objetos JSON que permita la identificación de usuarios, cambio de estados (Activo, Ausente, Ocupado), envío de mensajes privados y públicos, así como la gestión de salas de chat dinámicas. El desarrollo siguió el modelo de cascada.

2. Análisis

En esta fase se definieron las capacidades que la implementación del sistema debe ofrecer según el protocolo.

2.1. Requisitos

A partir del análisis del protocolo, se identificaron los siguientes componentes:

- **Gestión de Identidad (IDENTIFY):** El sistema debe permitir al usuario unirse con un nombre único (con máximo 8 caracteres). El servidor debe validar la existencia previa y notificar a otros usuarios la entrada de nuevos miembros.
- **Gestión de Estado (STATUS):** Los usuarios pueden alternar entre **ACTIVE**, **AWAY** y **BUSY**. Cada cambio debe disparar un evento **NEW_STATUS** para los demás clientes conectados.
- **Comunicación (TEXT, PUBLIC_TEXT y ROOM_TEXT):**
 - Mensajes privados que solo el destinatario recibe. Si el destinatario no existe, entonces debe regresar un error.
 - Mensajes públicos dirigidos a todos los usuarios en la conexión global.
 - Mensajes dirigidos a todos los usuarios en una sala. Si la sala no existe o el usuario no pertenece a ella se regresa un error.
- **Gestión de Salas (ROOMS):** El sistema debe permitir crear salas (con máximo 16 caracteres), invitar usuarios y gestionar pertenencia a sala. Se requiere validación de que el usuario esté en la sala antes de realizar operaciones, que esté invitado antes de unirse o que la sala exista antes de realizar otras operaciones.
- **Obtención de usuarios (USERS y ROOM_USERS):** El usuario puede obtener la lista de usuarios que están conectados globalmente o en una sala a la que pertenece. Si la sala no existe o no está en ella, se regresa un error.

- **Desconexión (DISCONNECT):** Manejo de limpieza de recursos cuando un cliente pierde conexión o decide salir, notificando su salida a las salas que pertenece y globalmente.

Todo mensaje JSON debe terminar con el carácter de salto de línea. Cuando todos los usuarios de una sala la abandonen, la sala será eliminada. Un usuario siempre se conecta con el estado ACTIVE. Un usuario debe identificarse antes de cualquier operación. Un mensaje JSON mal formado regresa un error.

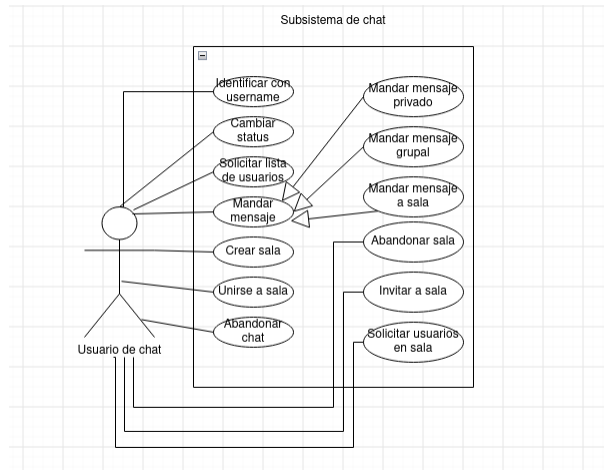


Figura 1: Diagrama del análisis (Use Case).

3. Diseño

El diseño implementa una arquitectura dividida en cliente y servidor con el objetivo de desacoplar las partes del programa y permitir que diferentes clientes puedan conectarse a un mismo servidor. El servidor actúa como un gestor de múltiples conexiones simultáneas. El cliente se encarga de la representación visual y la administración de la lógica local.

A grandes rasgos, antes de definir cualquier proceso, se define lo siguiente:

1. Una lista de usuarios global.
2. Una lista de salas global.
3. Algún tipo de estructura que guarde usuarios.
4. Algún tipo de estructura que guarde salas, una lista de miembros y una lista de usuarios invitados.
5. Hilos, refiriéndose a una forma de realizar operaciones con concurrencia.

Se definen vagamente debido a que, por ahora, solo son recursos abstractos que nos ayudarán a definir los algoritmos que necesita el protocolo. En la sección de desarrollo se definirán concretamente.

Algunos procesos que se detallan en el protocolo son simples, y a continuación se detalla el proceso:

1. **PUBLIC_TEXT:** El cliente manda la petición. Se itera sobre una lista de usuarios global y a cada usuario se le manda el mensaje.
2. **TEXT:** El cliente manda la petición. Se busca el usuario dentro de una lista de usuarios global. Si el usuario existe, se manda el mensaje. Se regresa un error en otro caso.
3. **USERS:** El cliente manda la petición. Se itera sobre una lista de usuarios global y se agrega cada usuario con su estado a un objeto JSON. Después, este objeto JSON se incluye al objeto JSON de respuesta y se regresa esta respuesta.
4. **ROOM_USERS:** El cliente manda la petición. Se verifica pertenencia a la sala especificada y se realiza la misma acción que en **USERS**, pero utilizando la lista de usuarios de la sala y no la global.
5. **LEAVE_ROOM:** El cliente manda la petición. Se verifica pertenencia y existencia de la sala. Si alguna de las dos condiciones es falsa, se regresa un error. En otro caso, se elimina al usuario de una sala de invitados y una sala de miembros asociadas a esa sala.

Para los procesos complejos, se describirán a continuación utilizando diagramas UML.

3.1. Diagramas UML

Se detallarán en diagramas UML la lógica usada para implementar el protocolo.

3.1.1. Diagrama de secuencia (Proceso de autenticación)

El siguiente diagrama representa la secuencia que sigue el cliente y el servidor cuando un usuario busca autenticarse.

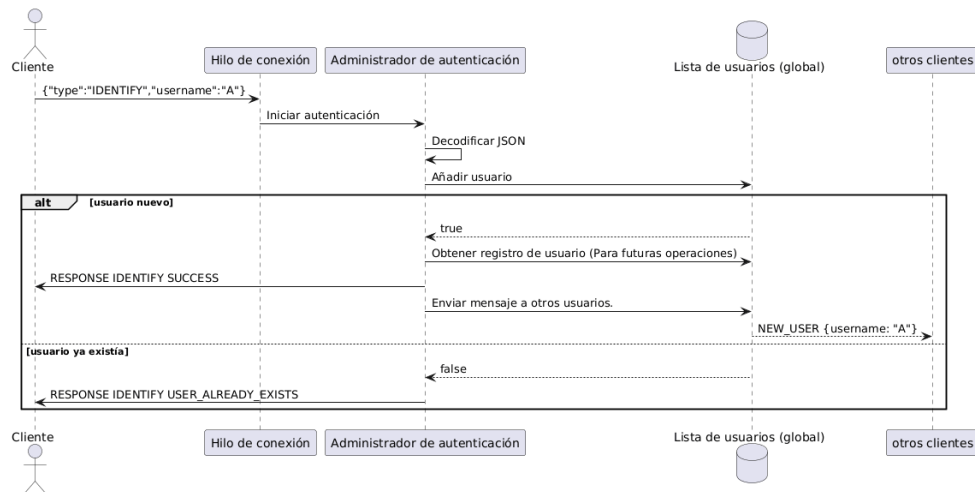


Figura 2: Diagrama de la autenticación.

3.1.2. Diagrama de secuencia (Crear sala)

El siguiente diagrama representa la secuencia que sigue el cliente y el servidor cuando un usuario crea una nueva sala.

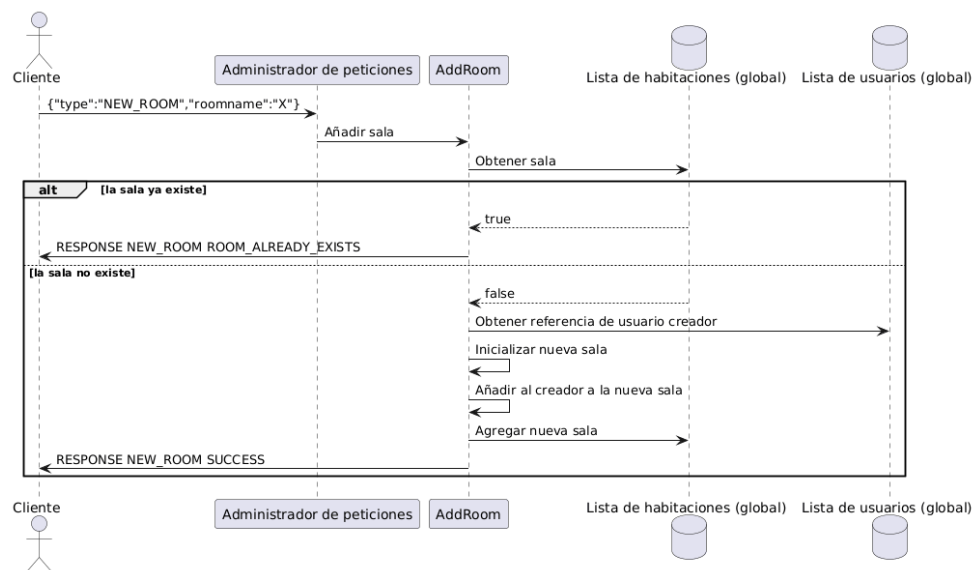


Figura 3: Diagrama de secuencia sobre la creación de una nueva habitación.

3.1.3. Diagrama de secuencia (Invitar y unirse a sala)

El siguiente diagrama representa la secuencia que siguen dos clientes y el servidor en el proceso de invitar y unirse a una sala.

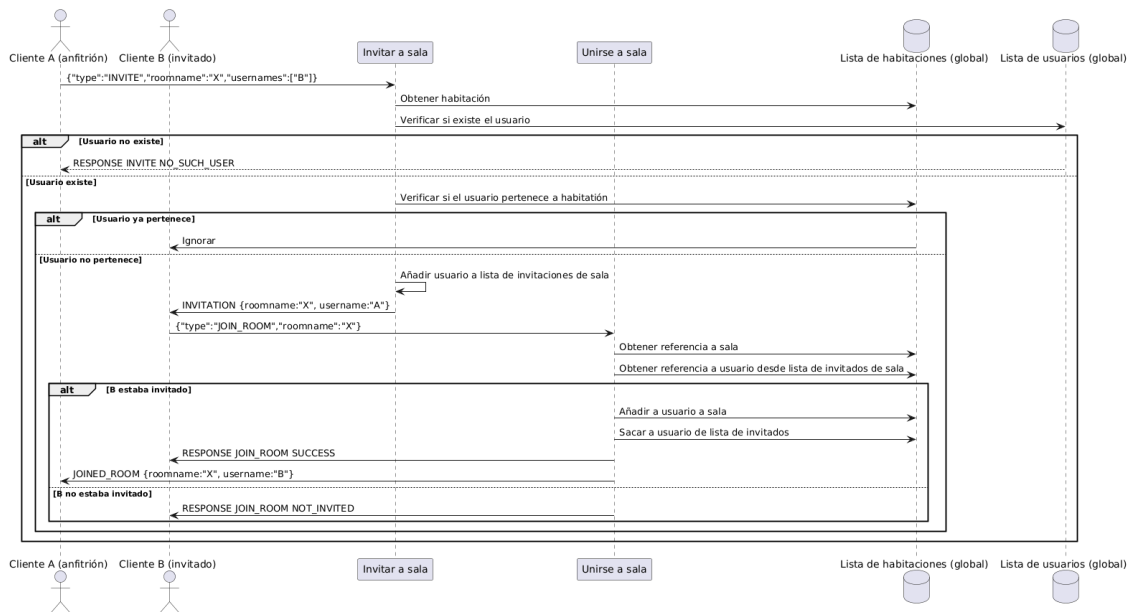


Figura 4: Diagrama de secuencia sobre el proceso de invitar y unirse a una sala.

3.1.4. Diagrama de secuencia (Texto a sala)

El siguiente diagrama representa la secuencia que siguen el cliente y el servidor al mandar un mensaje a una sala.

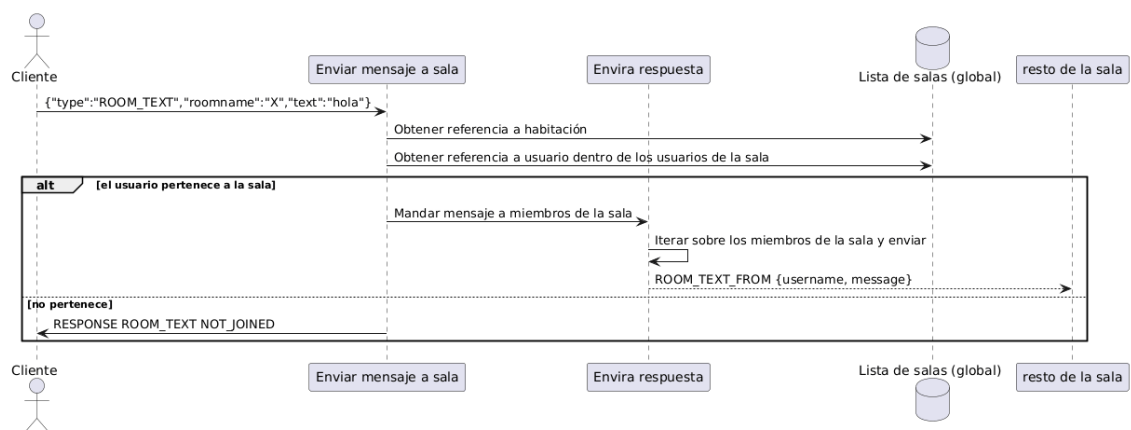


Figura 5: Diagrama de secuencia sobre el proceso de enviar un mensaje a una sala.

3.1.5. Diagrama de secuencia (Desconexión)

El siguiente diagrama representa la secuencia que siguen el cliente y el servidor al momento de desconectar al cliente.

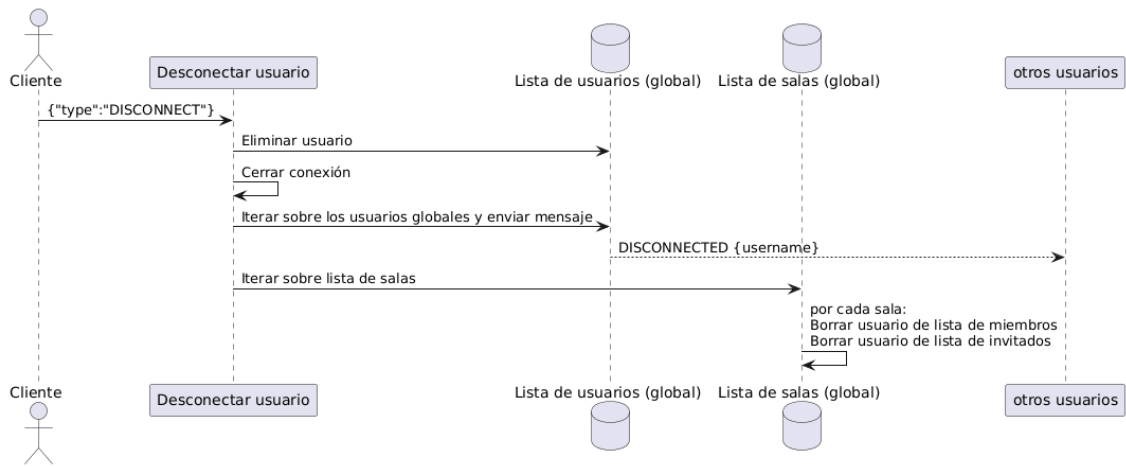


Figura 6: Diagrama de secuencia sobre el proceso de desconexión.

4. Desarrollo

La implementación del servidor se realizó en el lenguaje **C**. El cliente se realizó en el lenguaje **Kotlin** utilizando el framework **Compose Multiplatform**.

4.1. Servidor

Empezando con la parte del servidor, se presentaron múltiples desafíos debido a la simplicidad del lenguaje **C**.

El lenguaje **C** tiene que compilar, ensamblar y vincular cada archivo que escribamos. Para poder automatizar este proceso y hacerlo amigable para el usuario, se optó por usar el sistema de construcción **meson**, con el propósito de poder configurar un directorio de construcción y obtener el ejecutable principal con un solo comando. Luego, dado que el protocolo especifica una comunicación utilizando objetos JSON, este fue el primer desafío, ya que **C** no tiene una biblioteca estándar para crear objetos JSON. Para solucionar este problema, se optó por utilizar la biblioteca **cJSON**, que provee diversas funciones que facilitan la creación de objetos JSON, incluyendo las más básicas que requiere nuestro protocolo, como asociar una llave con un valor, que nos permite interpretar y crear la mayor parte de las peticiones y respuestas del protocolo; asociar una llave con otro objeto JSON, que nos ayuda al momento de mandar las listas de usuario; y determinar si un valor asociado a una llave es un arreglo, que nos permite lidiar con las invitaciones de forma sencilla.

Otro desafío que surgió derivado de la elección del lenguaje fue el manejo de cadenas de texto. **C** no tiene algún primitivo u objeto **String**, en realidad son arreglos de caracteres con un símbolo nulo al final. Esto causo muchos problemas, pues la costumbre ha sido manejar cadenas como en un lenguaje orientado a objetos. Se obtuvieron varios errores del tipo "Segmentation Fault".¹ erróneamente asumir que la declaración de **String** era equivalente a **char* string** y después hacer declaraciones del tipo **string = string2** dentro de las funciones, omitiendo que estas variables

viven dentro de la pila de ejecución y ese apuntador apuntará a otros datos después del término de la función.

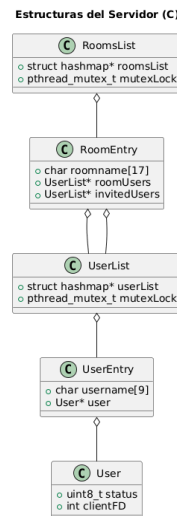
Otro error relacionado al manejo de cadenas fue el de utilizar directamente la función `cJSON_Print(json)`, pues se asumió que la función solo imprimiría la cadena correspondiente al objeto `json` sin hacer algo más, pero al correr el programa `valgrind` y al leer la documentación cuidadosamente se determinó que la función reservaba memoria en el heap, regresaba un apuntador a esa dirección y la debíamos liberar después.

El siguiente (y principal) desafío fue lidiar con la programación concurrente.

El servidor debe atender múltiples conexiones simultáneas. Para esto, se implementó la creación de hilos de ejecución para cada usuario utilizando la biblioteca `pthread.h`. Derivado de esto, necesitamos manejar el acceso simultaneo a los datos importantes, para lo cual utilizamos una estructura `pthread_mutex_t` para bloquear el acceso simultaneo.

La creación de un socket en el servidor, en pocas palabras, consiste de crear un file descriptor TCP sobre IPv4 (o IPv6, pero esta implementación utiliza solamente IPv4), crear una dirección del servidor que incluya el puerto, realizar la operación `bind`, realizar la operación `listen`, empezar a aceptar conexiones y crear un hilo para cada una.

Las principales estructuras en las cuales se basan los algoritmos del servidor son las siguientes:



Notemos que el diagrama de clases no tiene ningún método. Esto es porque C no es un lenguaje orientado a objetos, el cual conformó otro desafío que se solucionó escribiendo funciones que tomen esas estructuras y las modifiquen tomando en cuenta ciertas condiciones.

Las estructuras de listas utilizan una implementación de la estructura de datos `hashmap` que se configura para comportarse como un conjunto. Se eligió esta estructura de datos por su complejidad en tiempo $O(1)$ amortizada para obtener elementos, la cual es superior a la complejidad $O(n)$ de una lista, aunque en ciertos casos (como al iterar sobre el diccionario para mandar un mensaje a todos los usuarios) sigue siendo $O(n)$.

Debido a este uso de `hashmap` fue necesario separar los usuarios `User` de `UserEntry`, ya que la implementación de `hashmap` almacena apuntadores a los elementos que se almacenan, y necesitamos que un usuario tenga datos variables, principalmente el estado. Almacenando un apuntador a `UserEntry` y tomando como llave a `username`, obtenemos lo que necesitamos porque podemos simplemente dereferenciar el apuntador `User*` y cambiar su estado, sin tener que reemplazar el valor dentro de nuestro diccionario `hashmap`.

Un efecto secundario de los apuntadores `User*` fue reservar y manejar la memoria de cada usuario. Esto fue resuelto usando una de las funciones de la implementación de `hashmap`, que permite ejecutar una función de liberación de memoria cada que se elimine un elemento. Las salas fueron implementadas como una estructura con un nombre, una lista de usuarios miembros y otra lista de usuarios invitados con el fin de reutilizar las funciones que actúan sobre una `UserList`. Al crear las listas de usuarios para cada sala, surgió el mismo efecto secundario que se mencionó al hablar de los apuntadores `User*`, pero se solucionó de la misma forma.

Cabe resaltar que `User*` solo es liberado cuando se desconecta el usuario; lo anterior mencionado de la lista de salas solo libera la memoria de las listas de miembros y de las listas de invitados. El servidor usa el Modelo Vista Controlador de la siguiente forma:

- Modelo: Estructuras `UserList`, `RoomsList`, `UserEntry`, `RoomEntry`.
- Controlador: Funciones que leen del socket, decodifican el JSON y manda la respuesta correspondiente (principalmente la función `determineJSONResponse()` y sus funciones asociadas).
- Vista: El servidor no tiene vista propia, pues de esto se encargará el cliente, pero tiene logs y respuestas JSON que se imprimen en pantalla.

4.2. Cliente

Avanzando a la parte del cliente, el primer desafío fue la implementación de una interfaz gráfica.

Dentro de los componentes de **Compose** se tuvo que realizar la decisión de utilizar **Material Design 2**, pues la versión más actual, **Material Design 3**, mandaba advertencias de componentes experimentales al momento de compilar, y esto puede causar problemas posteriormente.

Después de elegir la versión del framework, se requirió aprender que cada función `@Composable` en Kotlin se corre múltiples veces y no solamente una vez, lo cual causaba múltiples problemas si se asumía que lo contrario. Además, las variables que utiliza Compose se declaran de una forma distinta, principalmente utilizando la característica *delegate* (que transfiere cierto funcionamiento a otra clase) con la palabra `by` y el lambda `remember { mutableStateOf() }` que provee Kotlin compose.

El principal desafío con la programación de esta interfaz gráfica fue familiarizarse con mandar instrucciones obtenidas a través de los componentes en la interfaz gráfica a otros lugares del programa en los que no podía interactuar ese componente. Esto

fue solucionado a través del uso de argumentos lambda. Para que esto funcionara, se implementó una sección principal del programa que administra las pantallas principales y los datos que se comparten entre ellas, definiendo el lambda correspondiente y pasándolo a la función que dibuja en pantalla las distintas pantallas del programa. Para comunicarse con el servidor, debemos construir un objeto JSON y mandarlo al servidor por medio de un cliente que implemente un socket TCP. La parte de construcción de un JSON, a diferencia de C, es trivial si utilizamos la biblioteca **kotlinx.serialization**, con la cual podemos convertir una clase de datos (característica del lenguaje) a un JSON con una sola línea de código. La implementación de la clase de datos fue una clase **Message**. En lenguajes como **Java**, los atributos de la clase tendrían que ser definidos en la clase junto con los métodos constructores, *getters* y *setters*, pero el principal método constructor estaría muy cargado de información para la cantidad de atributos posibles que puede tener nuestro mensaje, así que se implementaría un patrón de diseño *builder*. Sin embargo, en Kotlin no es necesario este último patrón de diseño, pues solo hacemos necesario el atributo "type" hacemos que los demás sean opcionales con valor `null` por defecto (los cuales se ignorarán al momento de crear un JSON), ya que en Kotlin una clase puede ser instanciada con argumentos nombrados con el nombre del parámetro al que le queremos asignar ese valor. Utilizando la clase de datos Message fue bastante sencillo implementar la lectura de respuestas del servidor haciendo una función que utilice el control de flujo **when** que tiene Kotlin.

El cliente que maneja sockets TCP fue implementado usando corrutinas de Kotlin, ya que nos permiten mantener un ciclo para recibir respuestas del servidor sin interrumpir la ejecución del cliente, dado que la lectura bloqueante ocurre en **Dispatchers.IO**, y crucialmente no interrumpir el dibujo de la interfaz gráfica. Además, una de las principales razones para usar corrutinas en vez de hilos es que son una alternativa más ligera.

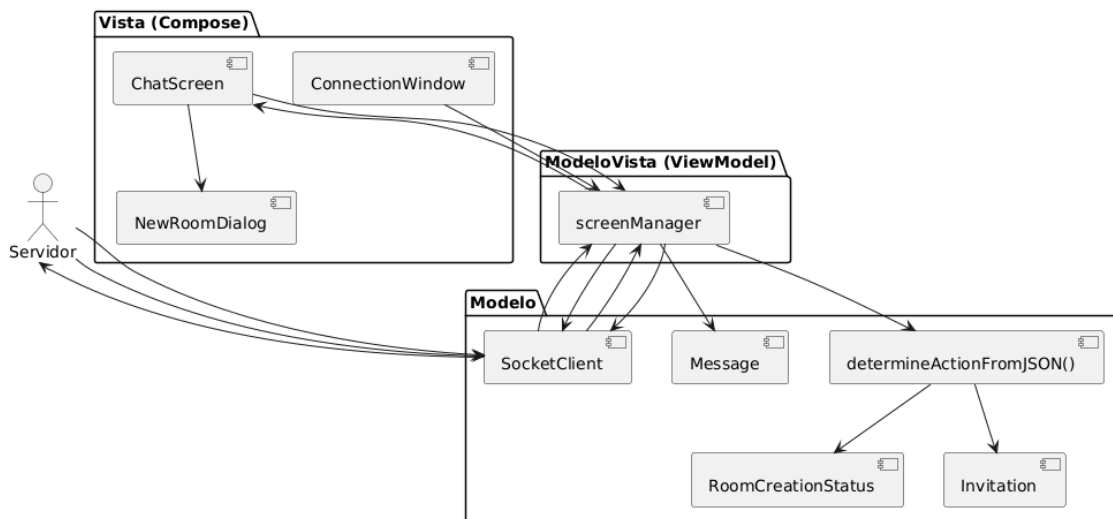


Figura 7: Diagrama de la arquitectura general del programa.

5. Conclusiones

Realizar este proyecto requiere conocimiento de múltiples ámbitos. Las principales cosas que se aprendieron fueron conceptos básicos de programación concurrente (Hilos y corrutinas), conceptos básicos de redes (Primeras capas del modelo OSI, sockets, protocolo TCP vs UDP, *Three-way handshake* en TCP, file descriptors en sistemas operativos unix, operación bind y listen, etc) y conceptos básicos de interfaces gráficas (Dibujar en pantalla, trabajar a fondo con lambdas, elementos del framework Compose, etc).

Personalmente, dejando aparte la discusión técnica, el proyecto fue difícil de realizar. Conocía los conceptos en un nivel superficial y nunca había trabajado con ellos. Aparte de los retos presentados en la fase de desarrollo, el principal reto fue al aprendizaje, pues tuve que aprender dos nuevos lenguajes (Kotlin y C), los conceptos básicos de Compose, programación concurrente y programación de sockets. El proyecto puede ser mejorado en muchos aspectos, principalmente garantizando que el servidor nunca puede caerse con peticiones maliciosas de usuarios y manejando todos los *memory leaks* que se pudieran presentar en la ejecución de todo el servidor. Además, el cliente utiliza funciones muy básicas del framework compose y no realiza las mejores prácticas de las interfaces gráficas (solamente consiste en botones, campos de texto y columnas de texto).

Otra mejora que ayudará en posteriores desarrollos es la implementación de pruebas unitarias no triviales. En este proyecto determiné que las pruebas unitarias tienen el potencial de tener una gran utilidad al hacer cambios y refactorizar código. Dentro de las primeras fases de desarrollo del servidor, implementé un hilo de ejecución para leer comandos del teclado y funciones para probar las listas de usuarios y para añadir usuarios inexistentes, pero esto puede ser considerado una mala práctica que sería mejor implementada en pruebas unitarias. La mayor parte de las pruebas se realizaron manualmente con una conexión a través de `netcat`, siguiendo manualmente lo que el protocolo especifica, lo cual tiene el potencial de ser automatizado y olvidarlo hasta que surja un error al cambiar lógica interior del programa.